



Модуль 9. Основи продуктивності та оптимізації

Урок 3. Швидкість API, HTTP- кешування та підготовка до highload

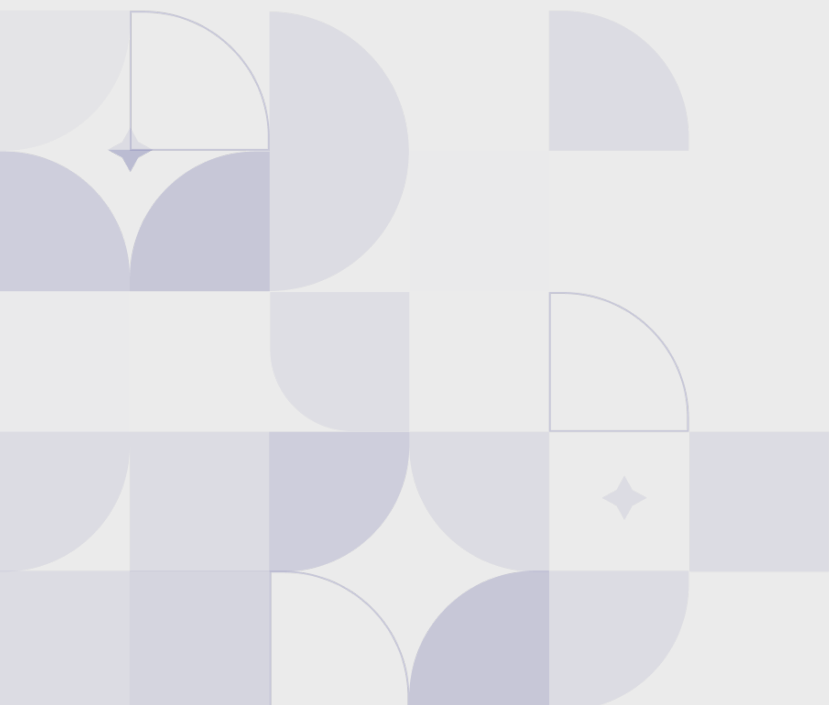
API як продукт: швидкість, стабільність, масштаб

Що ми вже маємо перед оптимізацією API

- Модулі 5–7: архітектура, DTO/Resources, REST-контракт, тести
- Модуль 8: логи, Sentry, лог-агрегація
- Модуль 9: кеш і БД (9.2) + HTTP-кеш, ліміти й highload (9.3)

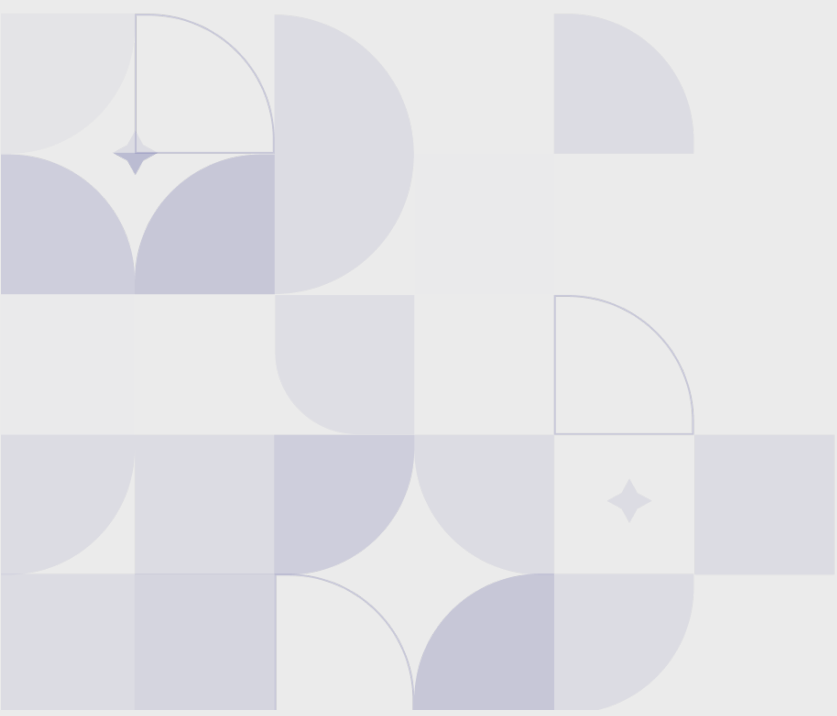
Мета уроку

1. розібрати, з чого складається час відповіді API
2. показати, як використовувати HTTP-кешування (Cache-Control, ETag)
3. наголосити на контролі об'єму даних (DTO/Resources, пагінація)
4. показати роль rate limiting і jobs за високого навантаження (highload)
5. зібрати фінальний образ production-ready API



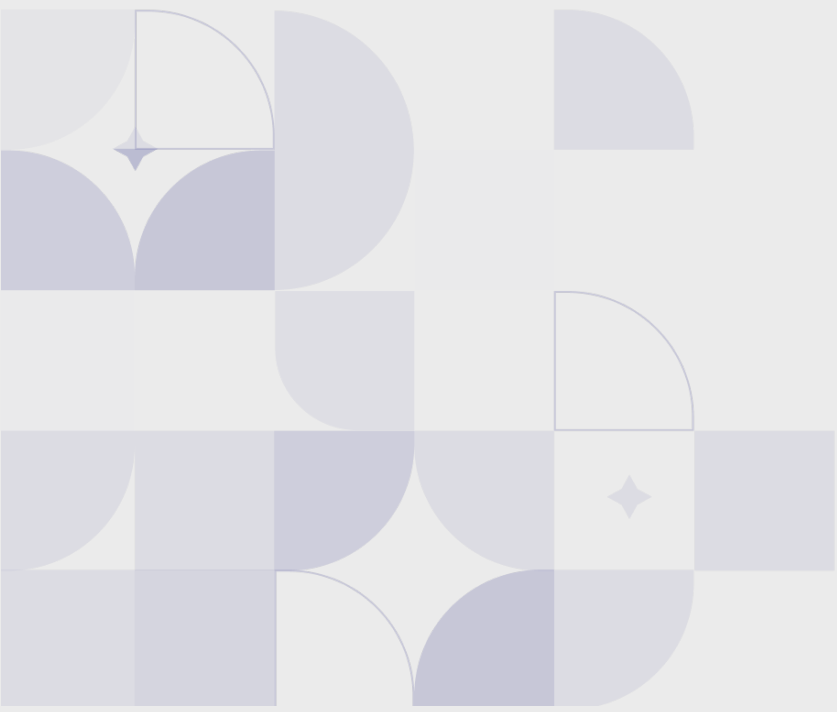
Повільне API = повільний продукт

- від API залежить веб, мобільний, інтеграції
- якщо бекенд дає 3 с — UX завжди 3 с
- у фінансах це ще й довіра: таймаути під час оплати $\approx$ втрата впевненості



Не один швидкий запит, а стабільний сервіс

- стабільні p95/p99 (без «піків»)
- немає деградації зі зростанням обсягу даних (пагінація, індекси)
- коректна робота під навантаженням (кеш, ліміти, jobs)



Шлях запиту в бекенді

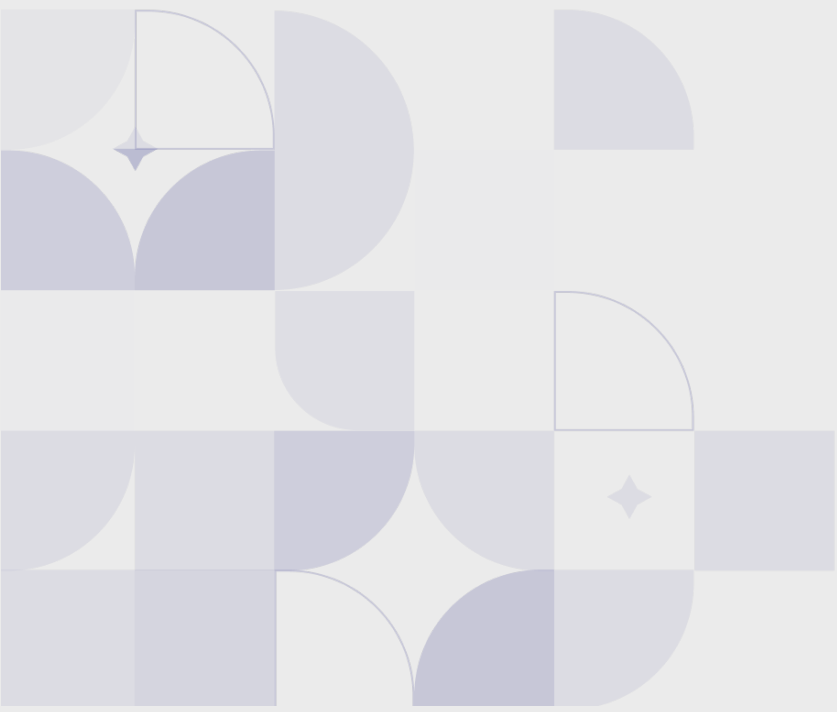
Етапи:

- мережа + прийом HTTP-запиту
- middleware (auth, логи, correlation_id)
- контролер/сервіси (бізнес-логіка)
- БД/кеш (основний «часовий» вклад)
- серіалізація JSON (DTO/Resources); відправка відповіді

Що дає нам HTTP

Можливості:

- Cache-Control, ETag, Last-Modified
- статус 304 Not Modified
- прокси/браузерний кеш



Кеш у клієнта/проксі, не в нашому коді

- ресурс кешується в браузері/CDN
- за повторного запиту — умовний запит (**If-None-Match/If-Modified-Since**)
- якщо не змінився — 304 без тіла

Основні заголовки HTTP-кешу

Cache-Control:

- **public, max-age=60** — можна кешувати де завгодно 60 с
- **private, max-age=300** — лише на клієнті 5 хв
- **no-store** — не кешувати зовсім

ETag:

- ідентифікатор версії відповіді
- з **If-None-Match** → 304, якщо не змінилось

Практика: Cache-Control для аккаунт-платежів і
довідників

Короткий private для персональних, довгий public
для спільних

Менше полів → менше байтів → швидше

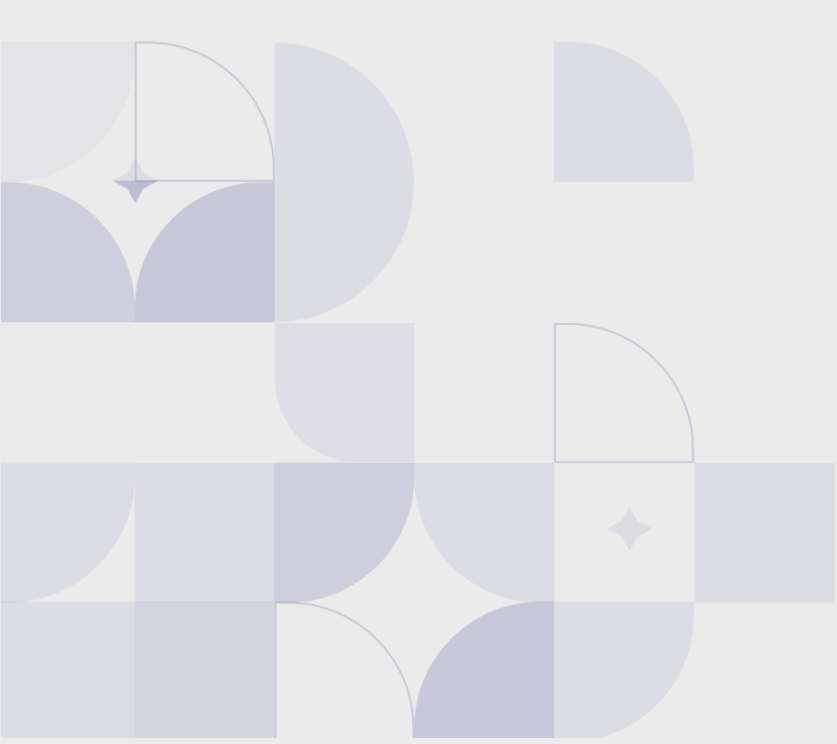
- DTO/Resources визначають набір полів
- зайві поля = зайвий трафік і серіалізація
- особливо критично для великих списків і звітів

Списки без пагінації недовго живуть

- обмежує розмір відповіді
- стабілізує навантаження
- must-have для платежів, транзакцій, звітів

Практика: рахуємо, що віддає endpoint

Скільки полів/байт і що можна прибрати



Ліміти запитів — захист для всіх

- обмежує частоту запитів від клієнта
- захищає від зловживань і надмірного навантаження з боку скомпрометованих клієнтів
- особливо важливий для публічних API й важких операцій

Не всі endpoint'и рівні ✨

Приклади:

- авторизація (захист від брутфорсу)
- експорт звітів/виписок
- heavy-операції з агрегації

Практика: обмежуємо частоту запитів до API

60 req/min для звичайних, 5 req/min для важких

Чого не варто робити в одному запиті

Приклади:

- генерація PDF-звіту
- побудова складної статистики за всіма рахунками
- масова обробка транзакцій

API швидко відповідає, воркер робить важке

Ідея:

- API приймає запит і додає job у чергу
- повертає 202 + task_id або посилання
- фронт опитує статус або отримує сповіщення

Практика: виносимо експорт виписки в job

Синхронно vs асинхронно

Не “вижати максимум з одного серверу”, а вміти масштабуватися

- кеш → менше навантаження на БД
- асинхронність → швидкі відповіді
- контроль обсягу й ліміти → передбачуване навантаження
- stateless API → можна додавати інстанси за балансувальником

Що потрібно для кількох екземплярів застосунку

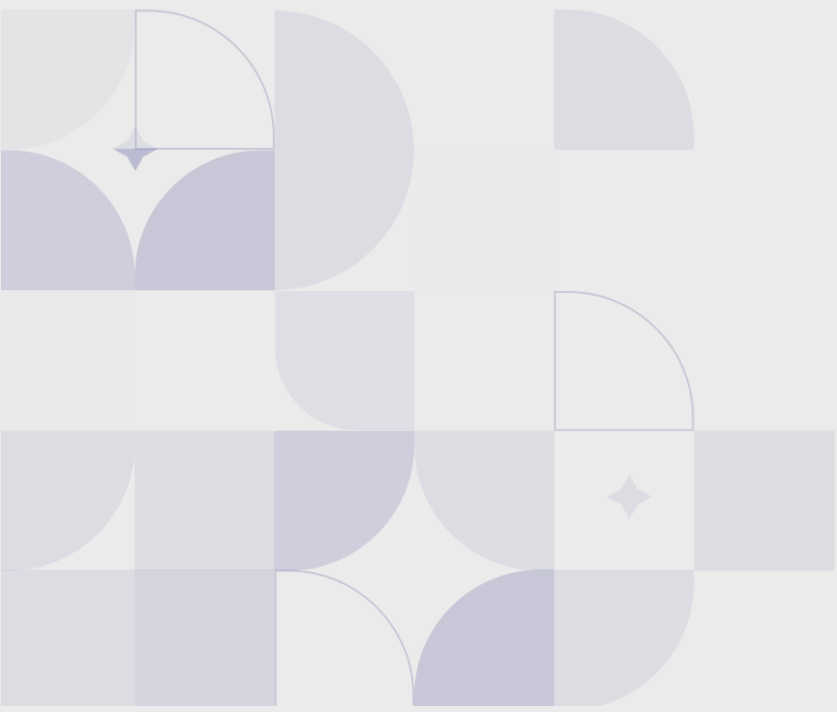
Вимоги:

- автентифікація за токеном/cookie без прив'язки до конкретного сервера
- сесії/кеш/черги в зовнішніх сховищах (Redis, БД)
- відсутність локального стану, від якого залежать запити

Як можна марно ускладнити життя собі й команді

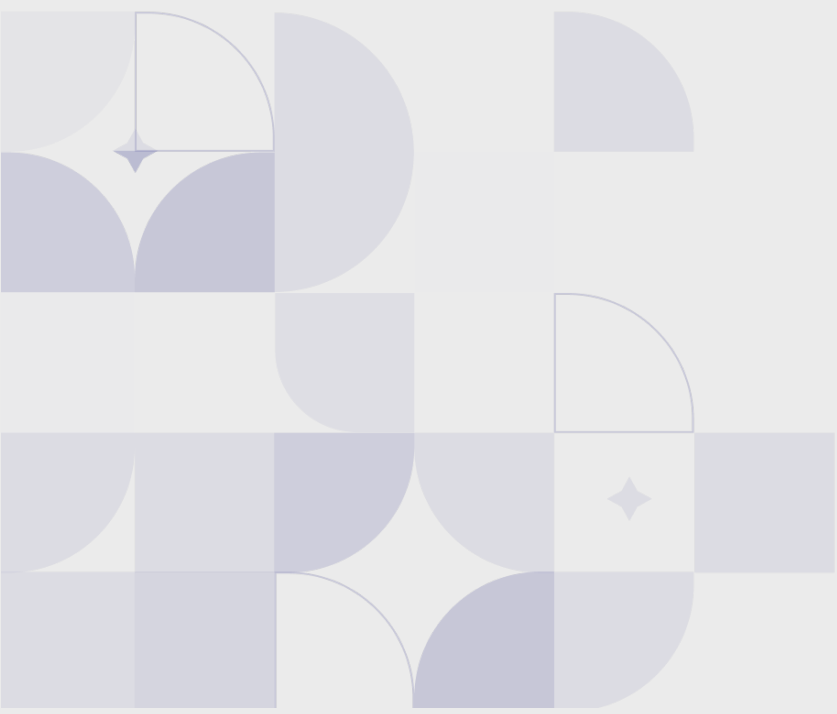
Приклади:

- складні рішення без вимірювань
- мікросервіси «на майбутнє»
- ігнорування HTTP-кешу й пагінації
- відсутність rate limiting
- «віддавати все й одразу» в одній відповіді



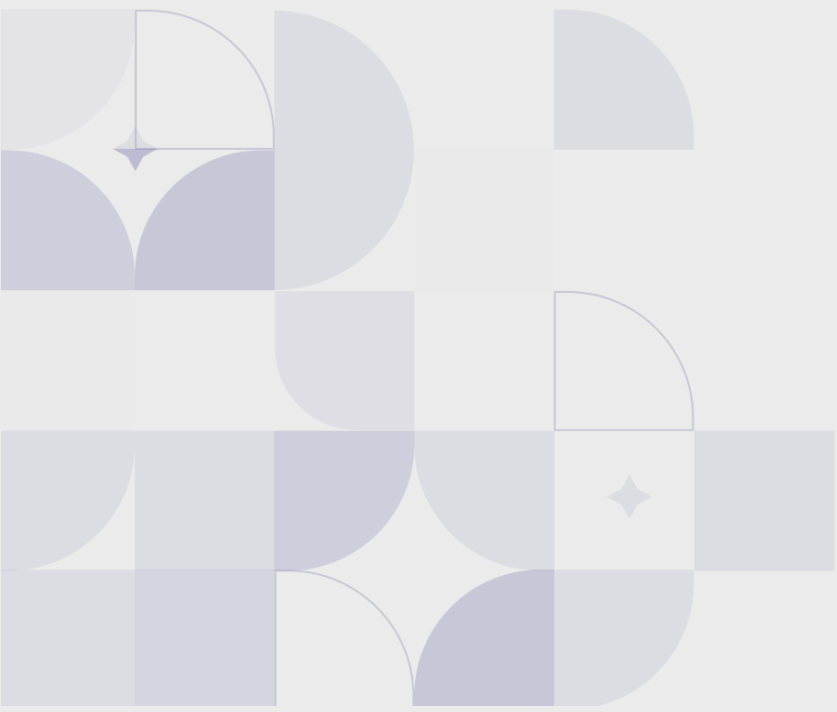
Що має бути в «живому» API

- оптимізована БД (індекси, відсутність N+1)
- кеш (серверний, де потрібно) + HTTP-кеш для ресурсів, що читаються часто
- пагінація всіх списків
- rate limiting для захисту
- jobs для важких операцій
- тести, логування, Sentry, лог-агрегація



Що ви тепер умієте

- Модулі 1–4: PHP/ООП/БД/Laravel
- Модуль 5: асинхронність, jobs
- Модуль 6: архітектура, якість коду, тести
- Модуль 7: REST API, DTO/Resources, документація, API-тести
- Модуль 8: логи, Sentry, лог-агрегація, інциденти
- Модуль 9: продуктивність (кеш, БД, HTTP-кеш, highload)



Production-ready бекенд — це не одна технологія

**Надійний фінансовий бекенд = архітектура +
чистий код + тести + API-контракт + логи/
моніторинг + продуктивність. Кожен елемент
важливий; разом вони дають систему, якій
можна довіряти**

Дякуємо за увагу, питання?